TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

TESI DI LAUREA MAGISTRALE IN INFORMATICA

Il Problema del Neighborhood Search su Grafi Temporali

Franco Salvucci - *0368692*

Relatore: Prof. Luciano Gualà

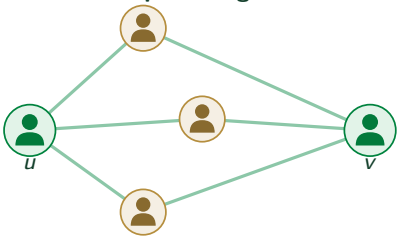
Correlatore: Dott. Maestro Alessandro Straziota

Università degli Studi di Roma "Tor Vergata" A.A. 2025/2026

Il contesto: utenti e documenti con vicinati simili

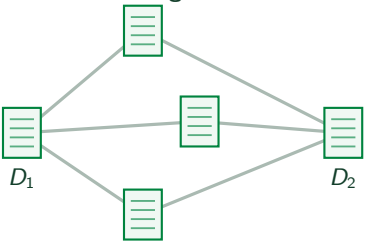
Le relazioni di un vertice descrivono interessi o comportamenti: cerchiamo vertici con **contatti simili**.

Rete sociale: profili seguiti in comune



Due utenti seguono tre profili uguali

Articoli: bibliografia in comune



Due articoli citano le stesse fonti

Il vicinato è l'insieme dei profili seguiti o degli articoli citati.

Neighborhood Search statico: intersezione e Jaccard

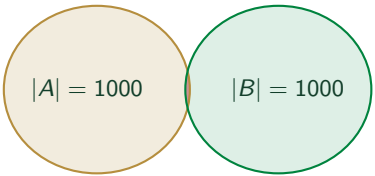
- **Input:** $G = (V, E)$, $\tau \in [0, 1]$ e $\sigma : 2^V \times 2^V \to [0, 1]$.
- **Query:** $S \subseteq V$.
- **Goal:** trovare $u \in V$ tale che $\sigma(S, N(u)) \geq \tau$.

Neighborhood Search statico: intersezione e Jaccard

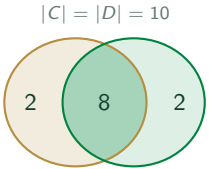
- **Input:** $G = (V, E)$, $\tau \in [0, 1]$ e $\sigma : 2^V \times 2^V \to [0, 1]$.
- **Query:** $S \subseteq V$.
- **Goal:** trovare $u \in V$ tale che $\sigma(S, N(u)) \geq \tau$.

$\sigma =$ **intersezione**

Insiemi grandi, sovrapposizione minima



Insiemi piccoli, sovrapposizione elevata



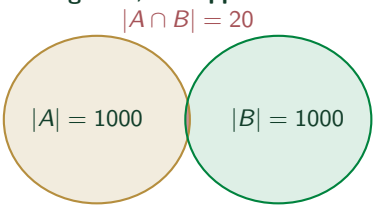
Quale coppia di insiemi risulta piu' simile?

Neighborhood Search statico: intersezione e Jaccard

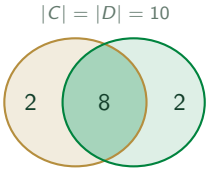
- **Input:** $G = (V, E)$, $\tau \in [0, 1]$ e $\sigma : 2^V \times 2^V \to [0, 1]$.
- **Query:** $S \subseteq V$.
- **Goal:** trovare $u \in V$ tale che $\sigma(S, N(u)) \geq \tau$.

$\sigma =$ **intersezione**

Insiemi grandi, sovrapposizione minima



Insiemi piccoli, sovrapposizione elevata



$|C \cap D| = 8$

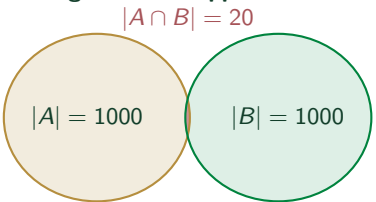
$20 > 8$: **l'intersezione premia gli insiemi grandi, senza normalizzare per la loro dimensione.**

Neighborhood Search statico: intersezione e Jaccard

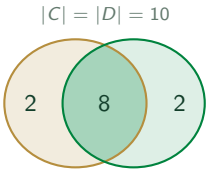
- **Input:** $G = (V, E)$, $\tau \in [0, 1]$ e $\sigma : 2^V \times 2^V \to [0, 1]$.
- **Query:** $S \subseteq V$.
- **Goal:** trovare $u \in V$ tale che $\sigma(S, N(u)) \geq \tau$.

$\sigma =$ **Jaccard**

Insiemi grandi, sovrapposizione minima



Insiemi piccoli, sovrapposizione elevata



$|C \cap D| = 8$

$J(C, D) = \frac{8}{12} \simeq 0.67$

$J(A, B) \simeq 0.01 < 0.67 \simeq J(C, D)$: **Jaccard riconosce la maggiore somiglianza relativa.**

Dalla ricerca esaustiva al filtro LSH

Schema *filter-and-verify*



Vantaggio

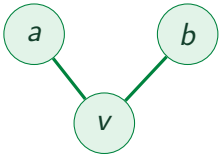
Evitiamo il confronto con tutti i vertici.

Compromesso

Il filtro è probabilistico: possibili falsi negativi.

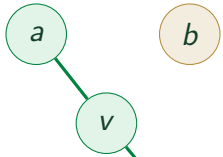
Le reti reali cambiano nel tempo

Istante t_1



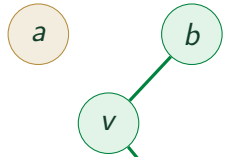
$N^{t_1}(v) = \{a, b\}$

Istante t_2



$N^{t_2}(v) = \{a, c\}$

Istante t_3



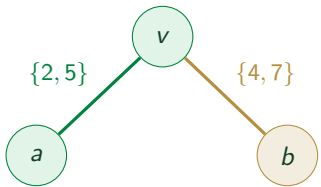
$N^{t_3}(v) = \{b, c\}$

Aggregare tutto in un grafo statico fa perdere l'informazione temporale.

Grafo temporale: definizione ed esempio

$\mathcal{G} = (V, E, \lambda)$: $\lambda(e)$ è l'insieme degli istanti associati all'arco e. Un evento è la tripla $(u, v, t) \in \mathcal{E}$.

Etichette temporali sugli archi



Attività del nodo v

$$\Lambda(v) = \{2, 4, 5, 7\}$$

Vicinato su un intervallo I

$$N^I(v) = \bigcup_{t \in I} N^t(v)$$

$$N^{[2,4]}(v) = \{a, b\}$$
$$N^{[5,6]}(v) = \{a\}$$

Due varianti del Neighborhood Search temporale

Input: $\mathcal{G} = (V, E, \lambda)$, $\tau \in [0, 1]$ e $\sigma : 2^V \times 2^V \to [0, 1]$, $\boldsymbol{\mu \geq 0}$.

Query: un nodo $v \in V$.

Goal: trovare $u \neq v$ e $I = [t, t + \mu]$ tali che $J(N^I(v), N^I(u)) \geq \tau$.

VARIANTE 1

Ampiezza fissa

μ nell'**input del problema**

VARIANTE 2

Ampiezza variabile

Due varianti del Neighborhood Search temporale

Input: $\mathcal{G} = (V, E, \lambda)$, $\tau \in [0, 1]$ e $\sigma : 2^V \times 2^V \to [0, 1]$.

Query: un nodo $v \in V$ e $\boldsymbol{\mu \geq 0}$.

Goal: trovare $u \neq v$ e $I = [t, t + \mu]$ tali che $J(N^I(v), N^I(u)) \geq \tau$.

VARIANTE 1

Ampiezza fissa

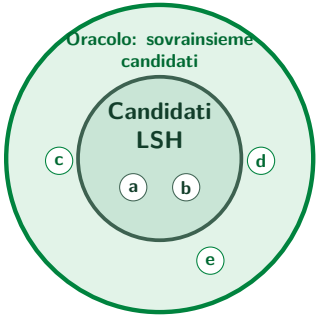
VARIANTE 2

Ampiezza variabile

μ fornita con la **query**

Contributo 2: un filtro per ampiezze variabili

- Indicizzare ogni ampiezza richiederebbe $\Theta(T^2)$ intervalli per nodo.
- **Euristica**: usare la prima componente MinHash per generare candidati.
- Rappresentazione geometrica, senza enumerare tutte le finestre.
- Nessun bound non banale sul numero di candidati, ma **pochi in pratica**.



$\{a, b\} \subseteq \{a, b, c, d, e\}$

SEZIONE 02

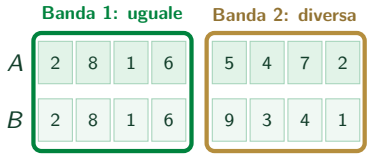
Variante 1: soluzione tecnica

LSH unificato, oracolo $\mathcal{O}$ e pruning temporale

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

LSH nel dettaglio: bande e tabelle hash

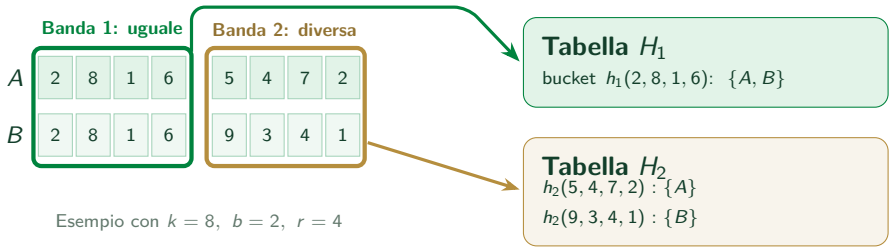
Lo sketch MinHash contiene $k = br$ valori; ogni **banda** (di r righe) va in una delle b tabelle hash.



Esempio con $k = 8,\ b = 2,\ r = 4$

LSH nel dettaglio: bande e tabelle hash

Lo sketch MinHash contiene $k = br$ valori; ogni **banda** (di r righe) va in una delle b tabelle hash.



Esempio con $k = 8,\ b = 2,\ r = 4$

LSH nel dettaglio: bande e tabelle hash

Lo sketch MinHash contiene $k = br$ valori; ogni **banda** (di r righe) va in una delle b tabelle hash.

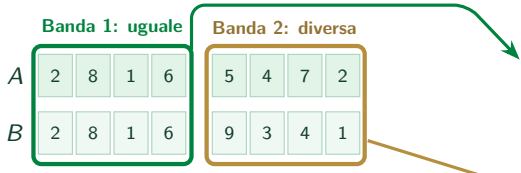


Tabella H_1
bucket $h_1(2, 8, 1, 6)$: $\{A, B\}$

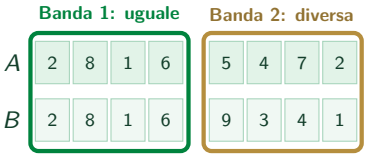
Tabella H_2
$h_2(5, 4, 7, 2) : \{A\}$
$h_2(9, 3, 4, 1) : \{B\}$

Esempio con $k = 8,\ b = 2,\ r = 4$

Collisione in $H_1 \Rightarrow$ coppia candidata

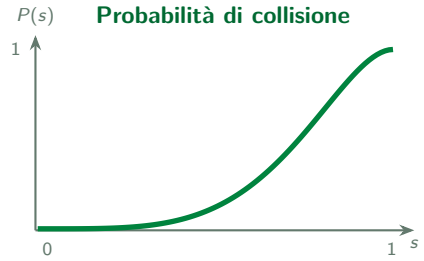
LSH nel dettaglio: bande e tabelle hash

Lo sketch MinHash contiene $k = br$ valori; ogni **banda** (di r righe) va in una delle b tabelle hash.

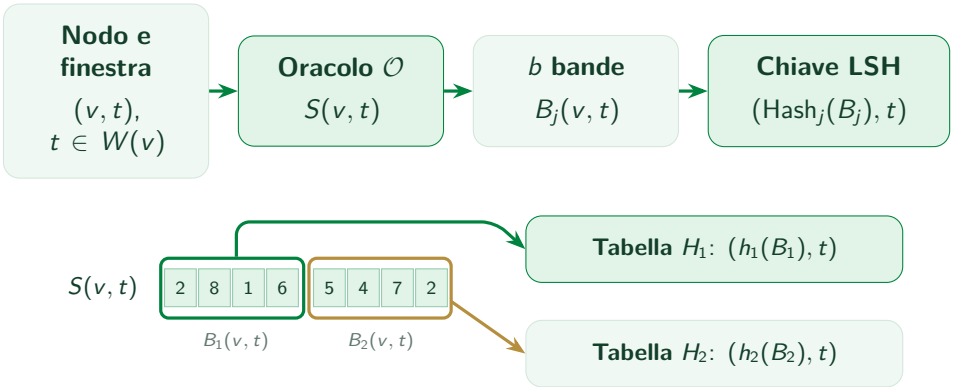


Esempio con $k = 8,\ b = 2,\ r = 4$

$$P(s) = 1 - (1 - s^r)^b$$



Adattamento temporale: b tabelle condivise



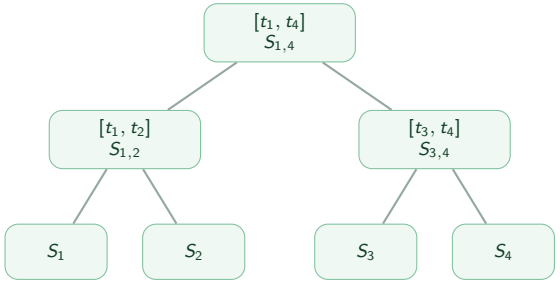
Uniche b tabelle per ogni μ fissata

Tutti i nodi e gli inizi di finestra condividono le tabelle.

Timestamp incluso nella chiave

Firme uguali in finestre diverse finiscono in bucket differenti.

Oracolo $\mathcal{O}$: recupero efficiente degli sketch



Query $[t_2, t_4]$: cerchiamo i nodi che coprono il range

Range Tree

sulle date attive $\Lambda(v)$

Sketch MinHash

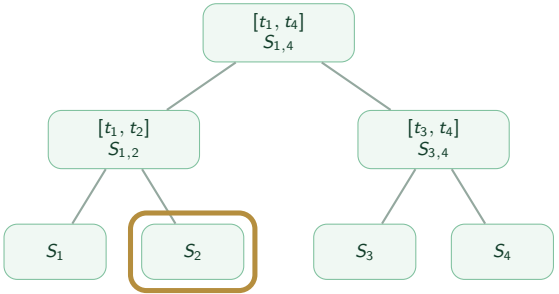
componibili via minimi

$O(k \log |\Lambda(v)|)$

Recupero dello sketch

Oracolo $\mathcal{O}$: building $O(k \sum_v m_v \log m_v)$; spazio $O(s \sum_v m_v)$ per sketch di dimensione s.
Con 1-MinHash, una RMQ recupera lo sketch in $O(1)$ (spazio $O(T)$ per nodo).

Oracolo $\mathcal{O}$: recupero efficiente degli sketch



Primo blocco selezionato: S_2

Range Tree

sulle date attive $\Lambda(v)$

Sketch MinHash

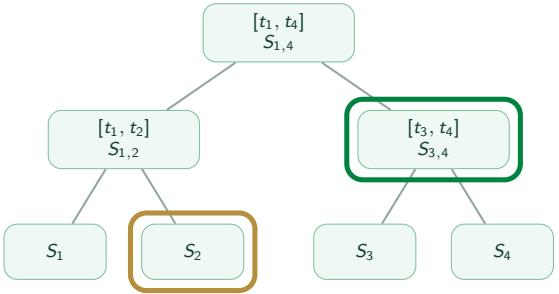
componibili via minimi

$O(k \log |\Lambda(v)|)$

Recupero dello sketch

Oracolo $\mathcal{O}$: building $O(k \sum_v m_v \log m_v)$; spazio $O(s \sum_v m_v)$ per sketch di dimensione s.
Con 1-MinHash, una RMQ recupera lo sketch in $O(1)$ (spazio $O(T)$ per nodo).

Oracolo $\mathcal{O}$: recupero efficiente degli sketch



Secondo blocco selezionato: $S_{3,4}$

Range Tree

sulle date attive $\Lambda(v)$

Sketch MinHash

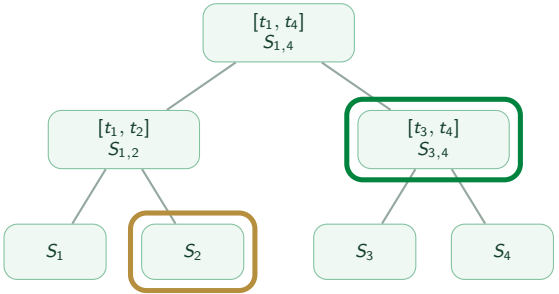
componibili via minimi

$O(k \log |\Lambda(v)|)$

Recupero dello sketch

Oracolo $\mathcal{O}$: building $O(k \sum_v m_v \log m_v)$; spazio $O(s \sum_v m_v)$ per sketch di dimensione s.
Con 1-MinHash, una RMQ recupera lo sketch in $O(1)$ (spazio $O(T)$ per nodo).

Oracolo $\mathcal{O}$: recupero efficiente degli sketch



Query $[t_2, t_4]$: merge di S_2 e $S_{3,4}$

Range Tree

sulle date attive $\Lambda(v)$

Sketch MinHash

componibili via minimi

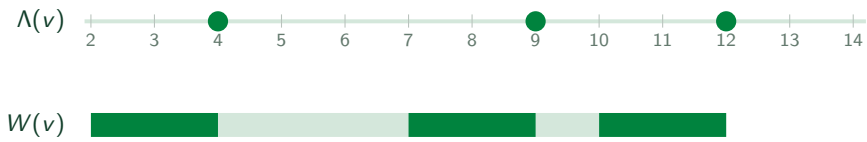
$O(k \log |\Lambda(v)|)$

Recupero dello sketch

Oracolo $\mathcal{O}$: building $O(k \sum_v m_v \log m_v)$; spazio $O(s \sum_v m_v)$ per sketch di dimensione s.
Con 1-MinHash, una RMQ recupera lo sketch in $O(1)$ (spazio $O(T)$ per nodo).

Pruning $W(v)$: meno finestre, stesso caso peggiore

$$W(v) = \{t : \ [t, t+\mu] \cap \Lambda(v) \neq \varnothing\}$$



Solo gli inizi di finestra che contengono almeno un evento

$$|W(v)| \leq \min\{T - \mu, (\mu + 1)|\Lambda(v)|\}$$

Il caso peggiore non migliora: se $|W(v)| = T - \mu$, si processano tutte le finestre, come con $T - \mu$ indici LSH statici. Il pruning è utile quando l'attività temporale è sparsa.

Bound teorici ottenuti

$w_v = |W(v)|$, $m_v = |\Lambda(v)|$ e $C_v = \sum_{t \in W(v)} |C(v,t)|$ candidati esaminati.

Costruzione indice

$$O\left(\sum_{v \in V} w_v(k \log m_v + b)\right)$$

Preprocessing delle b tabelle

Spazio indice

$$O\left(b \sum_{v \in V} w_v\right)$$

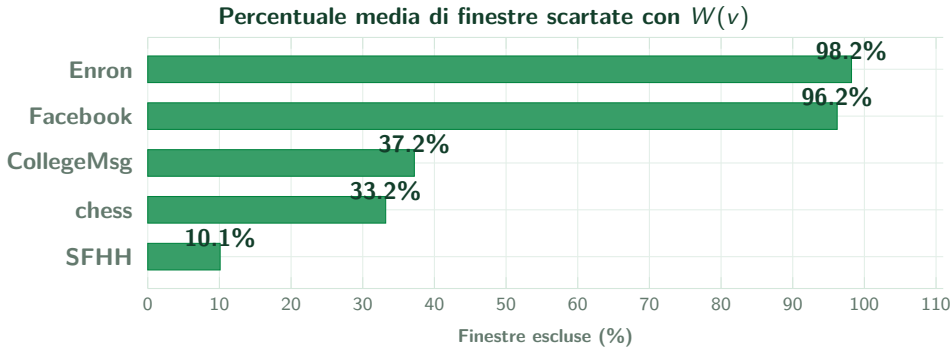
Bucket delle finestre indicizzate

Tempo di query

$$O(w_v(k \log m_v + b) + kC_v)$$

Inclusa la verifica delle firme

Il pruning nei dataset reali

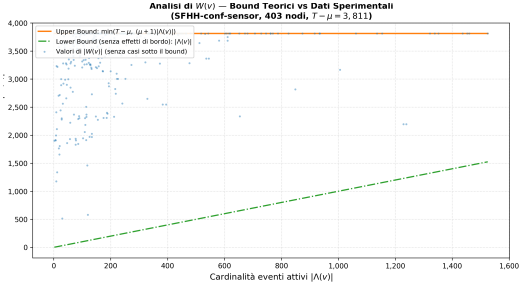


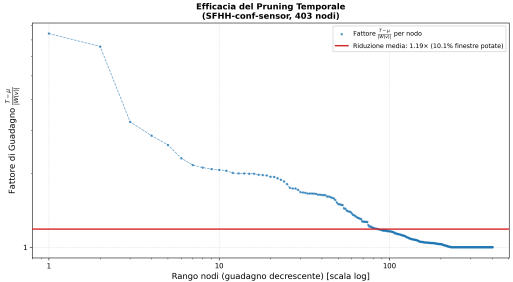
Enron: 98.2%

SFHH: 10.1%

Attività temporalmente sparsa: pruning molto efficace. Attività diffusa nel tempo: guadagno limitato.

Pruning su SFHH-conf-sensor: guadagno limitato



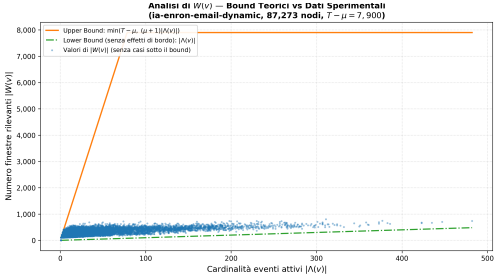


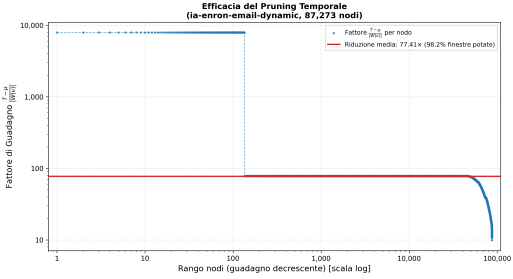
10.1% delle finestre eliminate

Guadagno medio 1.19×

Con eventi distribuiti su molti istanti temporali, $W(v)$ rimane vicino all'insieme delle finestre ammissibili.

Pruning su Enron-email: guadagno elevato





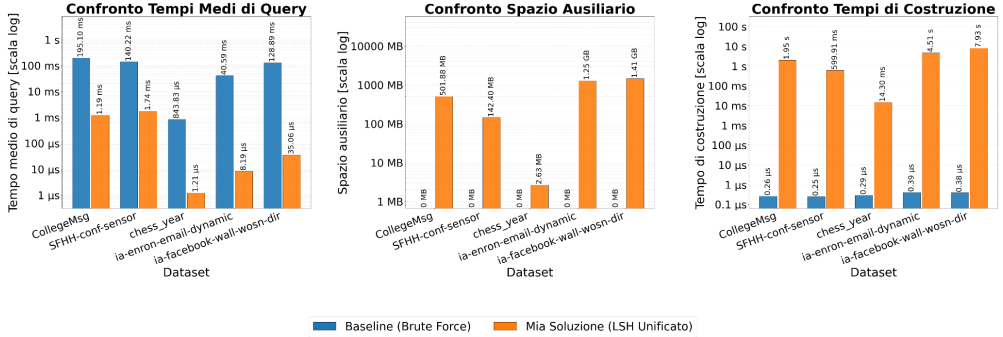
98.2% **delle finestre eliminate**

Guadagno medio 77.41×

La concentrazione degli eventi in pochi istanti consente di escludere molte finestre senza attività.

Prestazioni: Brute Force contro LSH+W

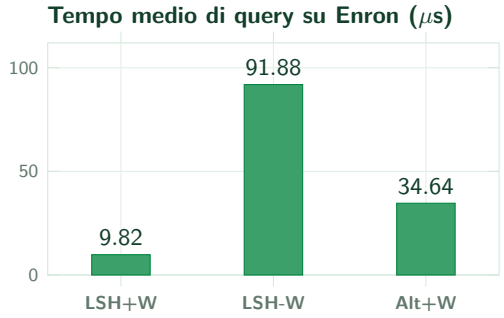
Parametri degli esperimenti: $k = 128, \quad b = 8, \quad r = 16, \quad \tau = 0.35.$



Enron: 8431× **più veloce** **Facebook:** 4287× **più veloce**

Enron: BF 82 757.90 μs, LSH+W 9.82 μs. La query booleana si arresta al primo esito positivo.

Qual è il contributo del pruning e dell'indice unico?



LSH+W

Indice unico con pruning.

LSH−W

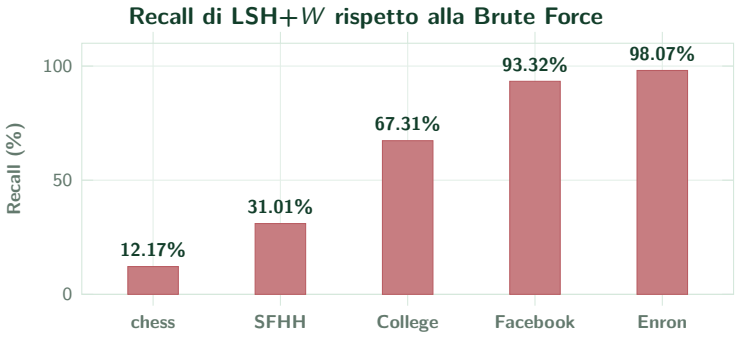
Stesso indice, senza pruning.

Alt+W

Indici indipendenti per finestra.

Il preprocessing e la memoria ausiliaria non sono trascurabili. Su SFHH il pruning non migliora il tempo di query.

Il compromesso probabilistico: la recall



Falsi negativi

Il filtro può scartare nodi con similarità sufficiente.

Recall

$$\frac{|B \cap L|}{|B|}$$

Confronto con BF sulle **medesime firme MinHash**. Zero falsi positivi rispetto alla BF nel campione.

SEZIONE 03

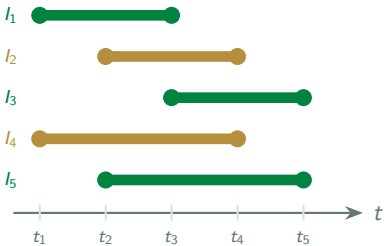
Variante 2: filtro geometrico

Una soluzione euristica per ampiezze variabili

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

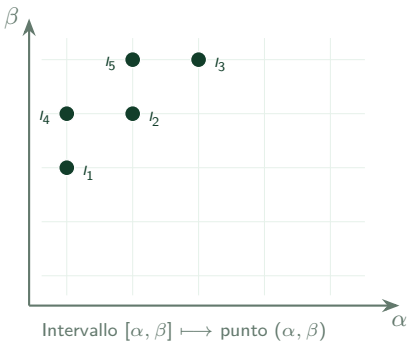
Dagli intervalli ai rettangoli geometrici

Intervalli temporali di esempio

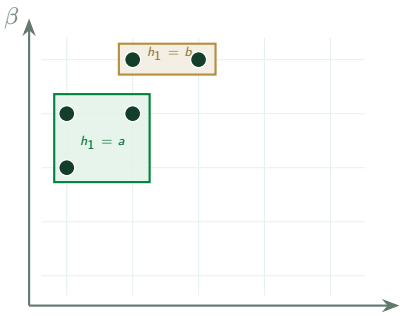


Ogni intervallo è definito da due estremi.

Dagli intervalli ai rettangoli geometrici



Dagli intervalli ai rettangoli geometrici



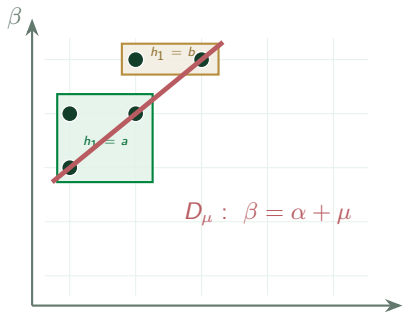
Stesso valore MinHash: stesso rettangolo.

Idea geometrica

Il primo MinHash resta costante su famiglie di intervalli rappresentabili con rettangoli.

$\Theta(T^2)$ intervalli $\longrightarrow$ al più T rettangoli per nodo.

Dagli intervalli ai rettangoli geometrici



La retta D_μ seleziona le finestre di ampiezza μ.

Idea geometrica

Il primo MinHash resta costante su famiglie di intervalli rappresentabili con rettangoli.

$\Theta(T^2)$ intervalli $\longrightarrow$ al più T rettangoli per nodo.

Query di ampiezza μ

Si cercano rettangoli con la stessa etichetta che intersecano la retta D_μ.

Costi teorici del filtro geometrico

M: coppie riportate dal filtro.

Building time

$$O\big(|\mathcal{E}| + nT \log^3 nT\big)$$

Spazio

$$O\big(nT \log^3 nT\big)$$

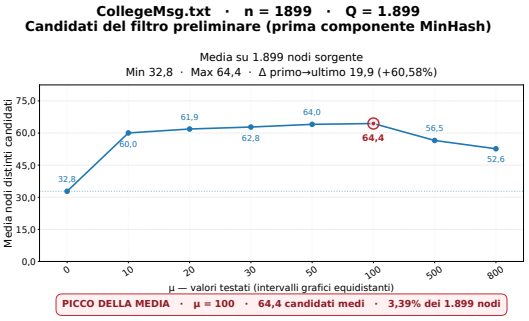
Identificare i candidati

$$O\big(T \log^3 nT + M\big)$$

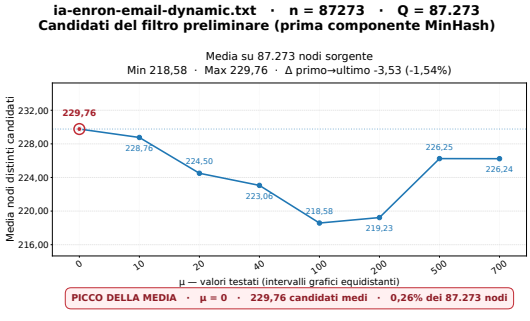
La verifica finale con le signature complete è esclusa da questi bound.

Non esiste un bound non banale sul numero di candidati M: la selettività del filtro dipende dai dati e dalla permutazione MinHash.

Il filtro euristico restituisce pochi candidati in pratica



32.8–64.4 candidati su 1 899 nodi



218.6–229.8 candidati su 87 273 nodi

Esperimento sul solo filtro $C_1(v, \mu)$. Non misura il costo della verifica, la recall o il tempo della query completa. I nodi ripetuti in più finestre si contano una sola volta.

SEZIONE 04

Conclusioni e sviluppi futuri

Contributi, limiti sperimentali e direzioni di ricerca

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Risultati principali e sviluppi futuri

Ampiezza fissa

Indice LSH unificato + pruning + oracolo $\mathcal{O}$

98.2%

finestre scartate su Enron

$8431\times$

speedup di query vs BF

Ampiezza variabile

Rettangoli + Range Tree geometrici

$O(T)$

gruppi di intervalli per nodo

Filtro selettivo sui dati sperimentati, da completare con la verifica.

Prossimi passi

Completare la variante 2,
studiare altre definizioni di Neighborhood Search.



Grazie per l'attenzione!

Franco Salvucci

Il Problema del Neighborhood Search su Grafi Temporali

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA